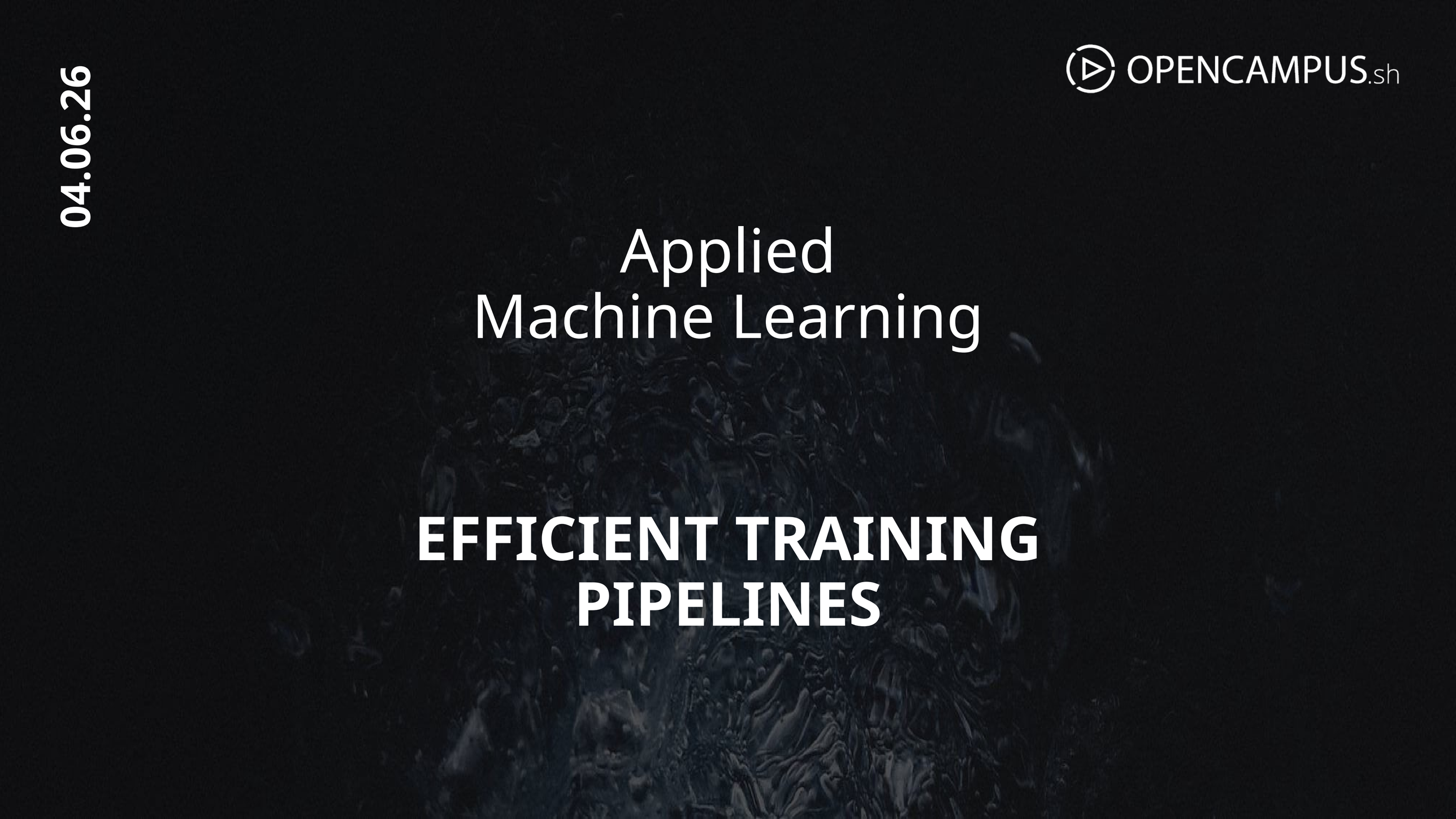
04.06.26

OPENCAMPUS.sh

Applied
Machine Learning

# EFFICIENT TRAINING PIPELINES



- QUIZ
- NEXT MILESTONE
- ASSIGNMENTS
- HF DATASETS LIBRARY
- TENSORBOARD

# QUIZ

https://forms.office.com/e/VWt0czuMAr

**16.04.2026** General Introduction
18:00 - 19:30
Gebäude 2.4, Fritz-Thiedemann-Ring 20, Heide + Kuhnkestr. 6, 24118 Kiel + ONLINE

Steffen Brandt
Referent:in
LinkedIn-Profil

**23.04.2026** Introduction into PyTorch
18:00 - 19:30
Gebäude 2.4, Fritz-Thiedemann-Ring 20, Heide + Kuhnkestr. 6, 24118 Kiel + ONLINE

Carl Schmidt
Referent:in
LinkedIn-Profil

**30.04.2026** Data Management and Neural Network Components
18:00 - 19:30
Gebäude 2.4, Fritz-Thiedemann-Ring 20, Heide + Kuhnkestr. 6, 24118 Kiel + ONLINE

Carl Schmidt
Referent:in
LinkedIn-Profil

**07.05.2026** Working with Text using Hugging Face
18:00 - 19:30
Gebäude 2.4, Fritz-Thiedemann-Ring 20, Heide + Kuhnkestr. 6, 24118 Kiel + ONLINE

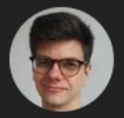
Jake Petersen
Referent:in

**14.05.2026** Feedback Sessions
18:00 - 19:30
Gebäude 2.4, Fritz-Thiedemann-Ring 20, Heide + Kuhnkestr. 6, 24118 Kiel + ONLINE

**21.05.2026** Hyperparameter Optimization
18:00 - 19:30
Gebäude 2.4, Fritz-Thiedemann-Ring 20, Heide + Kuhnkestr. 6, 24118 Kiel + ONLINE

Hannes Körner
Referent:in

**28.05.2026** Working with Images using TorchVision
18:00 - 19:30
Gebäude 2.4, Fritz-Thiedemann-Ring 20, Heide + Kuhnkestr. 6, 24118 Kiel + ONLINE



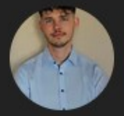
Hannes Körner
Referent:in

**04.06.2026** Efficient Training Pipelines
18:00 - 19:30
Gebäude 2.4, Fritz-Thiedemann-Ring 20, Heide + Kuhnkestr. 6, 24118 Kiel + ONLINE

Steffen Brandt
Referent:in
LinkedIn-Profil

**11.06.2026** Project Work
18:00 - 19:30
Gebäude 2.4, Fritz-Thiedemann-Ring 20, Heide + Kuhnkestr. 6, 24118 Kiel + ONLINE

**18.06.2026** Transformers for NLP in PyTorch
18:00 - 19:30
Gebäude 2.4, Fritz-Thiedemann-Ring 20, Heide + Kuhnkestr. 6, 24118 Kiel + ONLINE



Jake Petersen
Referent:in

**25.06.2026** Project Presentations - Part 1
18:00 - 19:30
Gebäude 2.4, Fritz-Thiedemann-Ring 20, Heide + Kuhnkestr. 6, 24118 Kiel + ONLINE

Steffen Brandt
Referent:in
LinkedIn-Profil

**02.07.2026** Project Presentations - Part 2
18:00 - 19:30
Gebäude 2.4, Fritz-Thiedemann-Ring 20, Heide + Kuhnkestr. 6, 24118 Kiel + ONLINE

# PROJECT MILESTONES

| | |
|---|---|
| 23.04. | Present your Ideas |
| 30.04. | Form Groups |
| 07.05. | Literature Review |
| 14.05. | Dataset Characteristics and Individual Feedback Sessions |

***Deadline for completing the repo sections: 17.05.***

| | |
|---|---|
| 21.05. | Definition of Model Evaluation |
| 28.05. | Project Work |
| 04.06. | Baseline Model Estimation |

***Deadline for completing the repo sections: 07.06.***

| | |
|---|---|
| 11.06. | Project Work / Optional Feedback Sessions<br>(*no regular class in this week!*) |
| 25.06. | Project Presentations, Part I |
| 02.07. | Project Presentations, Part II |

***Submission deadline for the documented repo: 01.08.2026***

# NEXT MILESTONE: BASELINE MODEL

- **How good is your final model actually? Metrics like RMSE or accuracy are difficult to interpret by themselves.**

- **A baseline model is a simple model that provides a context for the performance of your metric.**
  - In regression problems, this might be a linear model.
  - In time series problems, this might be a simple forecast of the last known value.
  - In certain tasks, it might be the performance of a human being.

- **It also requires you to think about metrics, data processing, train-test splits, which you need for the complex model as well.**

# PROJECT PRESENTATIONS

- Considering the content of the presentation follow the instructions given in the template repo here.
- Maximum Duration:
  The presentation must not exceed 15 minutes.
- All project members should participate.
- Please mark in the Excel with the project list whether you prefer the first or second presentation session or if you do not have a preference.

# ASSIGNMENTS

# ASSIGNMENTS NEXT SESSION?

# HUGGING FACE DATASETS LIBRARY

**Provides you with dataset level optimizations (the PyTorch Dataset library is focusing on samples):**

- **loading datasets from a dataset hub by name,**
- **automatic caching,**
- **dataset splits,**
- **metadata/dataset cards,**
- **streaming large public datasets with one flag,**
- **column-oriented transforms like .map(),**
- **Arrow-backed storage and efficient reuse.**

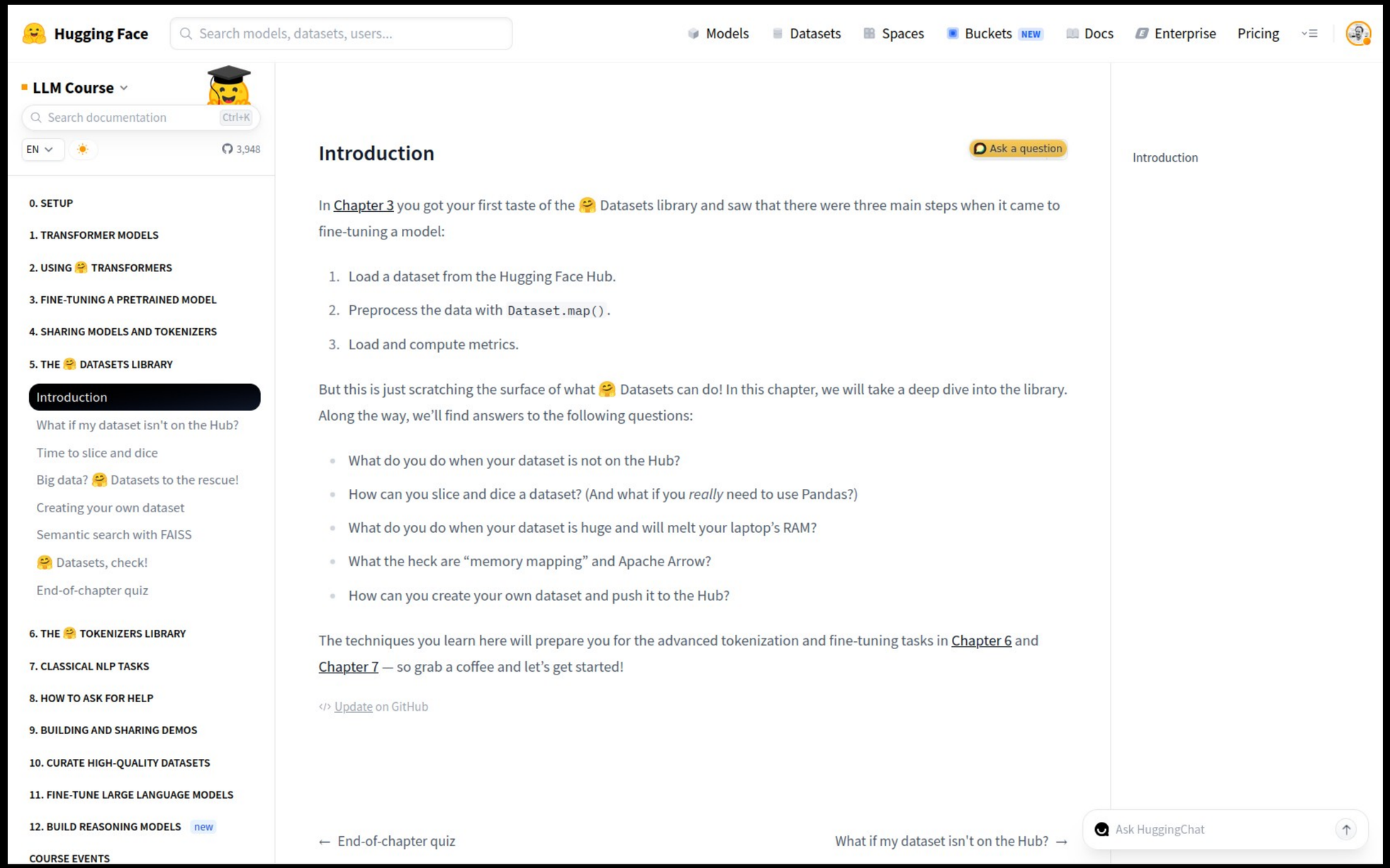
# EXAMPLES

```python
def add_word_count_batch(batch):
    batch["word_count"] = [len(t.split()) for t in batch["text"]]
    return batch

ds_mapped = ds.map(add_word_count_batch, batched=True)
```

```python
# Keep rows with more than one word
ds_short = ds.filter(lambda example: len(example["text"].split()) > 1)
```

# HF LM COURSE


Hugging Face
Search models, datasets, users...
Models
Datasets
Spaces
Buckets NEW
Docs
Enterprise
Pricing
LLM Course
Search documentation Ctrl+K
EN
3,948
0. SETUP
1. TRANSFORMER MODELS
2. USING 🤗 TRANSFORMERS
3. FINE-TUNING A PRETRAINED MODEL
4. SHARING MODELS AND TOKENIZERS
5. THE 🤗 DATASETS LIBRARY
Introduction
What if my dataset isn't on the Hub?
Time to slice and dice
Big data? 🤗 Datasets to the rescue!
Creating your own dataset
Semantic search with FAISS
🤗 Datasets, check!
End-of-chapter quiz
6. THE 🤗 TOKENIZERS LIBRARY
7. CLASSICAL NLP TASKS
8. HOW TO ASK FOR HELP
9. BUILDING AND SHARING DEMOS
10. CURATE HIGH-QUALITY DATASETS
11. FINE-TUNE LARGE LANGUAGE MODELS
12. BUILD REASONING MODELS new
COURSE EVENTS
Introduction
Ask a question
In Chapter 3 you got your first taste of the 🤗 Datasets library and saw that there were three main steps when it came to fine-tuning a model:
1. Load a dataset from the Hugging Face Hub.
2. Preprocess the data with Dataset.map().
3. Load and compute metrics.
But this is just scratching the surface of what 🤗 Datasets can do! In this chapter, we will take a deep dive into the library. Along the way, we'll find answers to the following questions:
What do you do when your dataset is not on the Hub?
How can you slice and dice a dataset? (And what if you really need to use Pandas?)
What do you do when your dataset is huge and will melt your laptop's RAM?
What the heck are "memory mapping" and Apache Arrow?
How can you create your own dataset and push it to the Hub?
The techniques you learn here will prepare you for the advanced tokenization and fine-tuning tasks in Chapter 6 and Chapter 7 — so grab a coffee and let's get started!
Update on GitHub
Introduction
← End-of-chapter quiz
What if my dataset isn't on the Hub? →
Ask HuggingChat

# TENSORBOARD

**TensorBoard:**

**“Show me curves and training diagnostics for this run.”**

**PyTorch Profiler:**

**“Explain why this run is slow.”**

**Weights & Biases:**

**“Track, compare, reproduce, and share all my runs.”**

# EXAMPLE (1)

bash

```
tensorboard --logdir=runs
```

python

```python
import torch
import torch.nn as nn
from torch.utils.tensorboard import SummaryWriter

# Initialize writer
writer = SummaryWriter("runs/experiment_1")

# Simple model
model = nn.Linear(10, 1)
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
loss_fn = nn.MSELoss()

# Training loop
for step in range(100):
    x = torch.randn(32, 10)
    y = torch.randn(32, 1)

    pred = model(x)
    loss = loss_fn(pred, y)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    # Log scalar metrics
    writer.add_scalar("Loss/train", loss.item(), step)

# Log model graph
dummy_input = torch.randn(1, 10)
writer.add_graph(model, dummy_input)

writer.close()
```

# EXAMPLE (2)

```bash
tensorboard --logdir=runs
```

```python
import torch
import torch.nn as nn
import lightning as L
from torch.utils.data import DataLoader, TensorDataset

class MyModel(L.LightningModule):
    def __init__(self):
        super().__init__()
        self.layer = nn.Linear(10, 1)
        self.loss_fn = nn.MSELoss()

    def forward(self, x):
        return self.layer(x)

    def training_step(self, batch, batch_idx):
        x, y = batch
        loss = self.loss_fn(self(x), y)
        self.log("train_loss", loss)  # ← auto-logged to TensorBoard
        return loss

    def configure_optimizers(self):
        return torch.optim.SGD(self.parameters(), lr=0.01)

# Dummy data
dataset = TensorDataset(torch.randn(320, 10), torch.randn(320, 1))
loader = DataLoader(dataset, batch_size=32)

# TensorBoardLogger is used by default, or be explicit:
from lightning.pytorch.loggers import TensorBoardLogger
logger = TensorBoardLogger("tb_logs", name="my_model")

trainer = L.Trainer(max_epochs=10, logger=logger)
trainer.fit(MyModel(), loader)
```

TensorBoard TIME SERIES SCALARS HPARAMS INACTIVE

Show data download links

Ignore outliers in chart scaling

Tooltip sorting method: default

Smoothing 0.6

Horizontal Axis

STEP RELATIVE WALL

Runs

Write a regex to filter runs

pneumonia_classifier/lr_1e-3

pneumonia_classifier/lr_1e-4

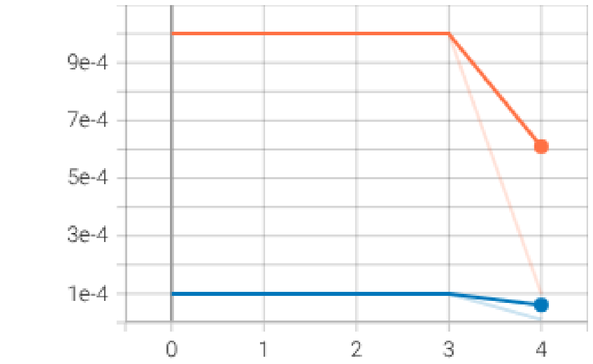
TOGGLE ALL RUNS

tb_logs

Filter tags (regular expressions supported)

lr-AdamW

lr-AdamW
tag: lr-AdamW




train_loss

train_loss
tag: train_loss

# TASKS UNTIL NEXT SESSION

- **Watch the videos of** *Module 3: Specialized Approaches to Natural Language Processing* **from the Pytorch for Deep Learning course**

- **Complete the assignment that will soon be given in the course handbook**

- **Estimate a baseline model for your data**